

Aether Auto-Update System

Architecture & Implementation Plan

In-Application Auto-Update for Aether Terminal Emulator
Without Reinstallation or Gatekeeper Re-Approval

Status: Proposed

Author: OKernel Architecture Team

Date: March 19, 2026

Version: 1.0

Classification: Internal Engineering Document

Repository: Vaiditya2207/OKernel

Component: apps/aether

Contents

1	Executive Summary	1
1.1	Key Principles	1
2	Why This Works Without Gatekeeper	1
3	System Architecture	1
3.1	Component Responsibilities	2
4	Backend Changes (SysCore)	2
4.1	New: GET /api/v1/aether/latest	2
4.2	New: GET /api/v1/aether/download/bundle	3
4.3	Upload Handler Update	3
4.4	CI/CD Change	3
5	Configuration	3
5.1	Config Schema	3
5.2	Config Fields	4
5.3	Swift Config Struct	4
6	Client-Side Architecture	4
6.1	UpdateManager	4
6.2	Version Comparison	5
6.3	Network Resilience	5
7	Update State Machine	6
7.1	State Transitions	7
8	Complete Update Flow	7
9	Install Sequence	9
9.1	Critical Path Steps	9
10	UI Components	10
10.1	Title Bar Layout	10
10.2	Badge Specification	10
10.3	Update Modal	11
10.4	State-Driven Modal Content	11
10.5	Post-Update “What’s New” View	11
11	File System Layout	12
12	Security Considerations	13
13	State Persistence	13
14	Implementation Phases	14
14.1	Phase 1: Foundation	14
14.2	Phase 2: Update Check	14
14.3	Phase 3: Title Bar Badge	14
14.4	Phase 4: Download + Modal	15

14.5 Phase 5: Install + Restart	15
14.6 Phase 6: Post-Update Experience	15
14.7 Phase 7: Polish + Tests	15
15 Test Plan	15
15.1 Unit Tests	15
15.2 Integration Tests	16
15.3 End-to-End Tests	16
15.3.1 test_full_update_lifecycle	16
15.3.2 test_update_with_disabled_config	16
15.3.3 test_update_rollback_on_failure	16
15.3.4 test_interrupted_download_cleanup	16
16 Risks and Mitigations	17
17 File Change Summary	17

1. Executive Summary

The system enables Aether to check for updates, download new versions, and apply them **without reinstalling** — avoiding macOS Gatekeeper re-approval. The update replaces the internals of the existing .app bundle in-place, requiring only an app restart. Users have full control via a config toggle in `~/.config/aether/config.json` or `config.toml`.

1.1 Key Principles

- **Zero friction** — one-click update from inside the app
- **No Gatekeeper** — in-place bundle patching, not a new install
- **User control** — config toggle to enable/disable entirely
- **Resilient** — backup before install, rollback on failure
- **Theme-aware** — all UI matches the user's chosen theme

2. Why This Works Without Gatekeeper

macOS Gatekeeper quarantine is applied to the **outer .app bundle** when first downloaded/opened. Once the user approves it, the `com.apple.quarantine` extended attribute is cleared. Replacing files *inside* the bundle (executables, frameworks, resources) does **not** re-trigger Gatekeeper — the approval lives on the .app directory itself.

Critical constraint: The bundle must be re-signed ad-hoc after patching. The `codesign` CLI is available on all macOS systems. This is the same technique used by Sparkle, VS Code, and iTerm2.

```
[YES] First install from DMG
      -> User approves once
[YES] In-place binary replace
      -> No re-approval needed
[YES] Ad-hoc re-sign after patch
      -> codesign --force --deep --sign -
[NO]  New .app from new DMG
      -> Gatekeeper triggers again
      (we avoid this)
```

Gatekeeper Behavior

3. System Architecture

The auto-update system spans three domains: the **SysCore backend** (artifact storage + API), the **Aether client** (check/download/install), and **GitHub Actions CI/CD** (build + publish).

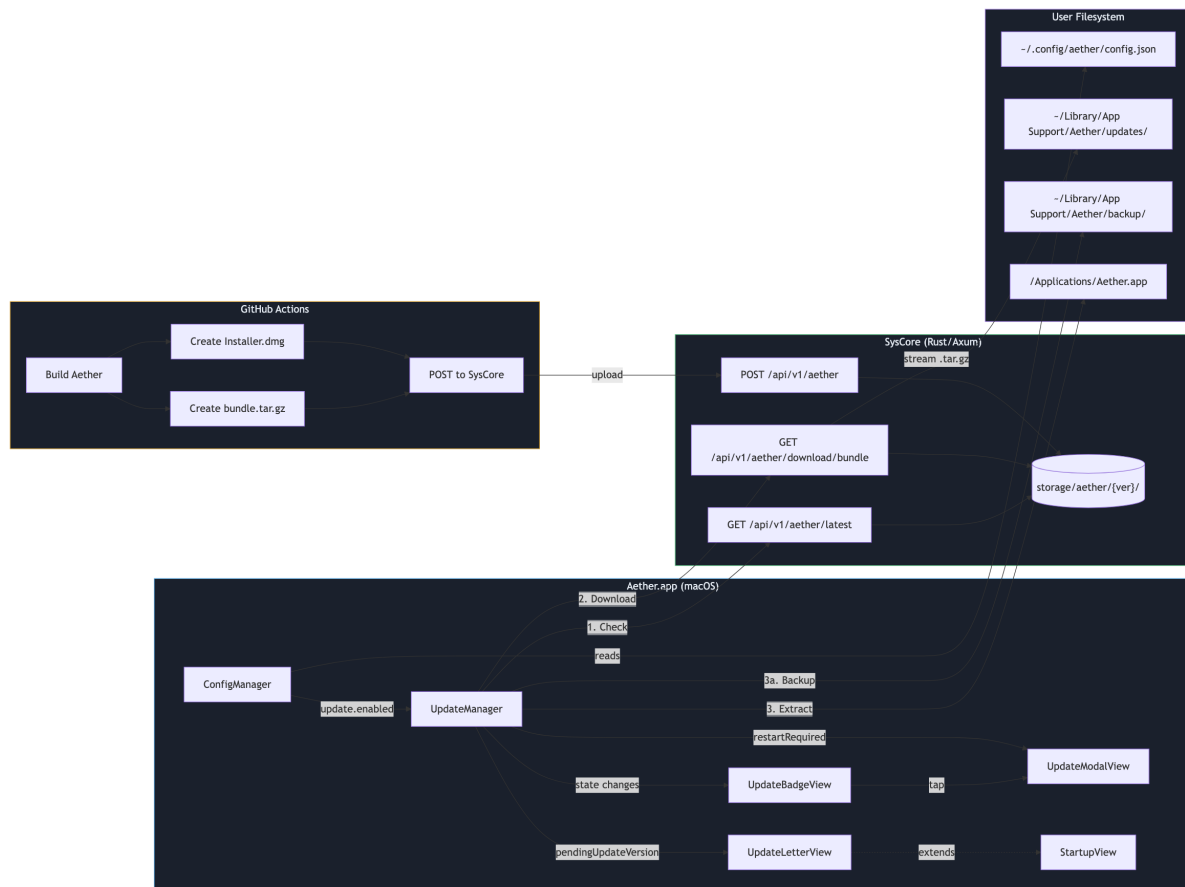


Figure 1: System architecture showing the three domains: Aether client, SysCore backend, and GitHub Actions CI/CD. Arrows indicate data flow for version checks, downloads, and artifact publishing.

3.1 Component Responsibilities

Component	Role
SysCore API	Serves version metadata and bundle tarballs
UpdateManager	Client-side singleton owning the full update lifecycle
UpdateBadgeView	Title bar indicator when update is available
UpdateModalView	Download/install progress overlay
UpdateLetterView	Post-update “What’s New” screen
ConfigManager	Reads <code>update.enabled</code> and settings
GitHub Actions	Builds <code>.tar.gz</code> bundles and publishes to SysCore

4. Backend Changes (SysCore)

4.1 New: GET /api/v1/aether/latest

Lightweight version-check endpoint. The existing `GET /api/v1/aether` returns ALL versions — too heavy for periodic polling.

File: `syscore/src/server/aether.rs`

```
{
  "version": "v0.4.0",
  "description": "Performance improvements",
}
```

```

"changelog": "- Fixed M3 rendering\n
              - 2x faster startup",
"release_date": "2026-03-19T00:00:00Z",
"size": 28456000
}

```

Response

Reuses `list_handlers` logic but returns only the first (latest by `release_date`) entry. Returns 404 if no versions exist.

Route registration in `lib.rs`:

```

.route("/api/v1/aether/latest",
  get(latest_handler))

```

4.2 New: GET /api/v1/aether/download/bundle

Serves a `.tar.gz` of the **app bundle contents** (not a DMG). A tarball can be extracted in-place — a DMG requires mounting and manual drag-to-Applications.

- CI/CD creates both DMG (fresh installs) and `Aether-bundle-{ver}.tar.gz` (updates)
- Tarball contains `Contents/` from inside `Aether.app/`
- Stored alongside DMG in `storage/aether/{version}/`

Query params: `?v=VERSION` or `?v=latest`

Response: Streamed application/gzip binary

4.3 Upload Handler Update

Modify `upload_handler` to accept an optional second file field `bundle` (the `.tar.gz`). The `metadata.json` gains:

```

{
  "version": "v0.4.0",
  "filename": "Aether_Installer.dmg",
  "size": 28456000,
  "bundle_filename":
    "Aether-bundle-v0.4.0.tar.gz",
  "bundle_size": 24000000
}

```

Extended Metadata

4.4 CI/CD Change

File: `.github/workflows/aether-release.yml`

After building the `.app` bundle:

```

cd build_artifact
tar -czf "Aether-bundle-${VERSION}.tar.gz" \
  -C Aether.app Contents

```

Upload both DMG and tarball in the multipart POST to SysCore.

5. Configuration

5.1 Config Schema

Add an update section to `AetherConfig`. Both JSON and TOML are supported.

```

{

```

```

"update": {
  "enabled": true,
  "check_interval": 3600,
  "server_url":
    "https://api.hackmist.tech",
  "channel": "stable"
}
}

```

JSON (/.config/aether/config.json)

```

[update]
enabled = true
check_interval = 3600
server_url = "https://api.hackmist.tech"
channel = "stable"

```

TOML (/.config/aether/config.toml)

5.2 Config Fields

Field	Type	Default	Description
enabled	bool	true	Master toggle
check_interval	int	3600	Seconds between checks (min: 300)
server_url	string	hackmist	SysCore base URL
channel	string	stable	Future: beta/nightly

5.3 Swift Config Struct

File: ConfigManager.swift

```

struct UpdateConfig: Codable {
    var enabled: Bool = true
    var checkInterval: Int = 3600
    var serverUrl: String =
        "https://api.hackmist.tech"
    var channel: String = "stable"

    enum CodingKeys: String, CodingKey {
        case enabled
        case checkInterval
            = "check_interval"
        case serverUrl = "server_url"
        case channel
    }
}

```

Add `var update: UpdateConfig = UpdateConfig()` to AetherConfig.

6. Client-Side Architecture

6.1 UpdateManager

File: Models/UpdateManager.swift

Singleton ObservableObject that owns the entire update lifecycle.

```

class UpdateManager: ObservableObject {
    static let shared = UpdateManager()

    @Published var state: UpdateState
        = .idle

```

```

@Published var availableVersion:
    AetherVersionInfo?
@Published var downloadProgress:
    Double = 0.0
@Published var statusMessage:
    String = ""

enum UpdateState {
    case idle
    case checking
    case available(String)
    case downloading
    case readyToInstall
    case installing
    case failed(String)
    case restartRequired
}

func checkForUpdates() { }
func downloadUpdate() { }
func applyUpdate() { }
func restartApp() { }
func schedulePeriodicCheck() { }
func cancelDownload() { }
}

```

6.2 Version Comparison

Semantic versioning comparison (major.minor.patch numerically). **Never string comparison.**

```

struct SemanticVersion: Comparable {
    let major: Int
    let minor: Int
    let patch: Int

    init?(_ string: String) {
        let cleaned = string
            .hasPrefix("v")
            ? String(string.dropFirst())
            : string
        let parts = cleaned
            .split(separator: ".")
            .compactMap { Int($0) }
        guard parts.count == 3
        else { return nil }
        (major, minor, patch) =
            (parts[0], parts[1], parts[2])
    }

    static func < (lhs: Self,
                    rhs: Self) -> Bool {
        (lhs.major, lhs.minor, lhs.patch)
        < (rhs.major, rhs.minor, rhs.patch)
    }
}

```

6.3 Network Resilience

- **Timeout:** 10s for version check, 120s for download
- **Background failure:** Silent retry at next interval
- **Manual check failure:** Error in modal
- **Download failure:** Error + retry button in modal

7. Update State Machine

The UpdateManager follows a strict state machine. Every UI element is driven by this state.

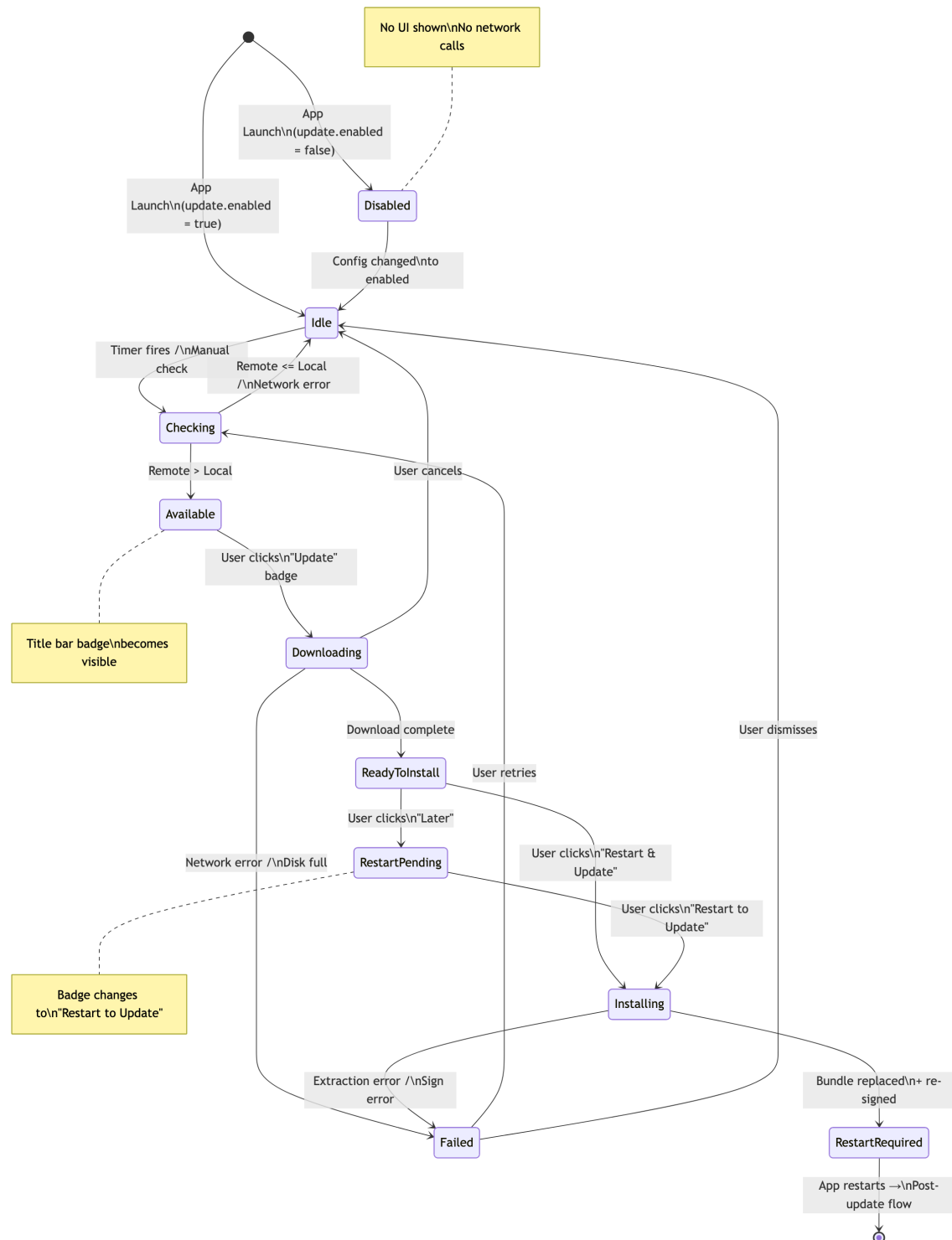


Figure 2: Update state machine. Transitions are triggered by timer events, user actions, and network responses. The Disabled state is reached when update.enabled = false in config.

7.1 State Transitions

From	To	Event
idle	checking	Timer / manual
checking	available	Remote > Local
checking	idle	Remote <= Local / error
available	downloading	User clicks badge
downloading	readyToInstall	100% complete
downloading	failed	Error
downloading	idle	User cancels
readyToInstall	installing	"Restart & Update"
readyToInstall	restartPending	"Later"
installing	restartRequired	Success
installing	failed	Error
failed	checking	Retry
failed	idle	Dismiss

8. Complete Update Flow

End-to-end flow from app launch to post-update experience:

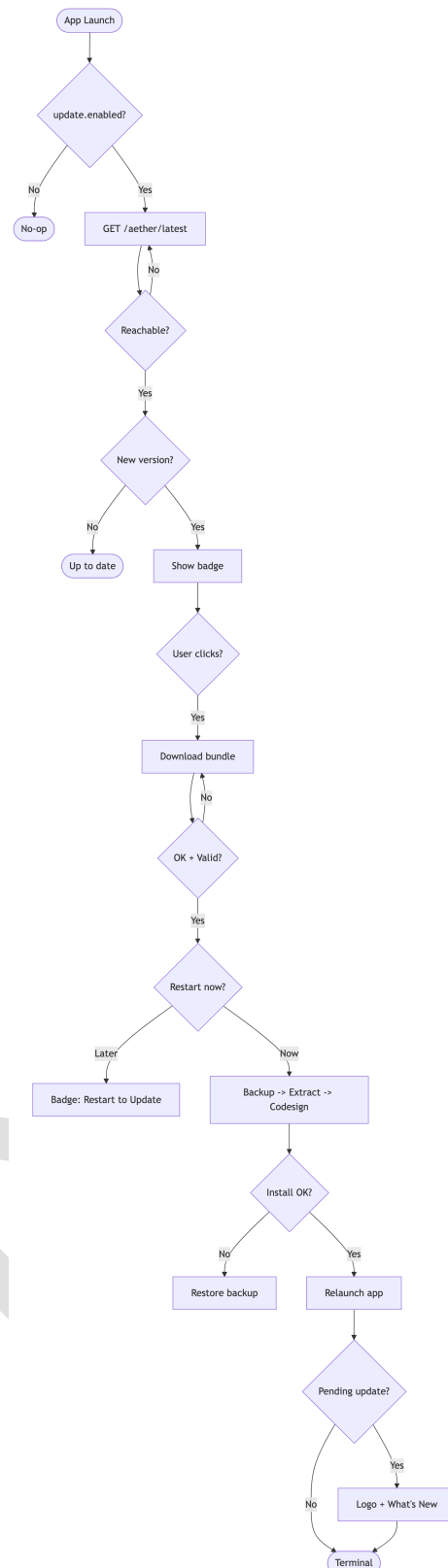


Figure 3: Complete update flow from app launch through version check, download, in-place install, and post-update “What’s New” experience.

9. Install Sequence

Detailed interaction between all components during the update process:

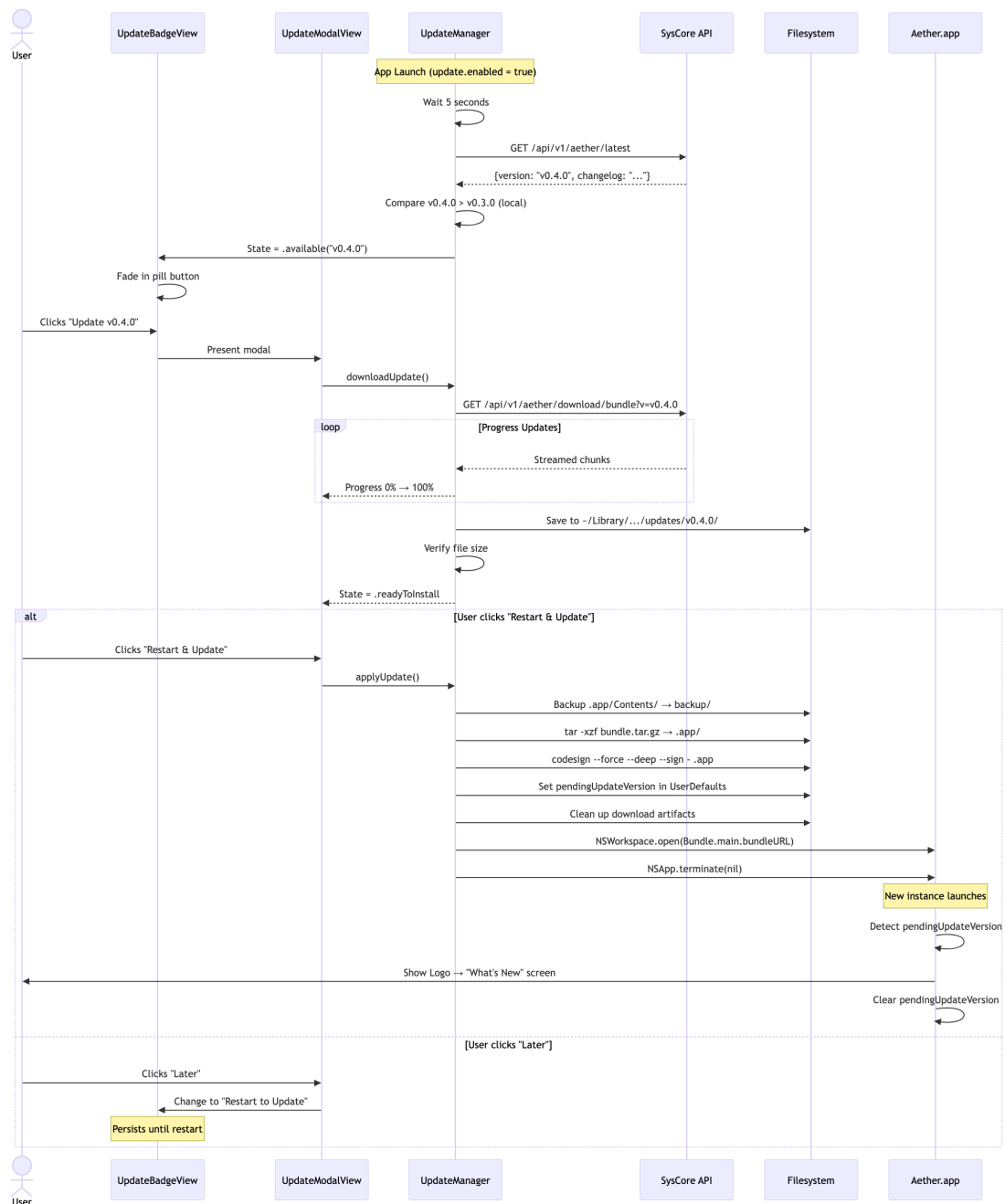


Figure 4: Sequence diagram showing interactions between User, UpdateBadgeView, UpdateModalView, UpdateManager, SysCore API, Filesystem, and Aether.app during the update life-cycle.

9.1 Critical Path Steps

1. Verify download integrity (file size matches `metadata.size`)
2. Locate current app bundle: `Bundle.main.bundlePath`
3. **Backup:** Copy `Contents/` to `~/Library/Application Support/Aether/backup/`
4. **Extract:** `tar -xzf {download} -C {bundle}/`

5. **Re-sign:** `codesign --force --deep --sign - {bundle}`

6. Set `pendingUpdateVersion` in `UserDefaults`

7. Clean up download artifacts

8. **Restart:** `NSWorkspace.open(bundleURL) + NSApp.terminate()`

Why tar via Process? Swift's built-in archive support is limited. `/usr/bin/tar` is on every macOS, handles `.tar.gz` natively. Run via `Process()`.

Why ad-hoc re-sign? Replacing the executable invalidates the code signature. Ad-hoc signing creates a local signature without a Developer ID. This is what the current `build_release.sh` already does.

Rollback: If step 4 or 5 fails, restore from backup immediately.

10. UI Components

10.1 Title Bar Layout

The update badge sits on the right side of the existing 28pt custom title bar:

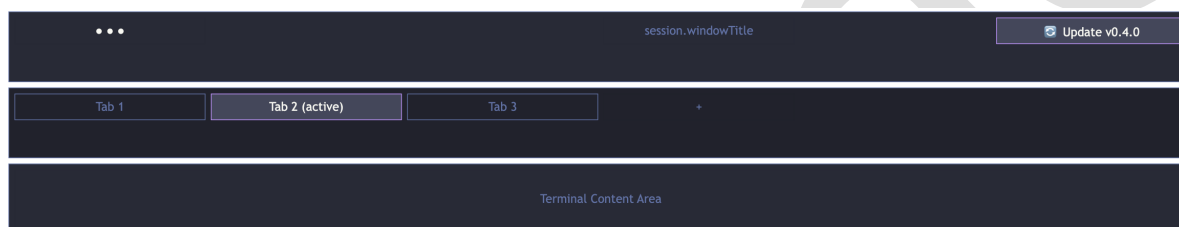


Figure 5: Title bar layout showing traffic lights (left), centered session title, and the update badge pill button (right). The tab bar sits below.

10.2 Badge Specification

Property	Value
Shape	Capsule (fully rounded pill)
Font	System 10pt, medium weight, monospaced
Padding	Horizontal 8pt, vertical 3pt
Background	Theme selection at 60% opacity
Text color	Theme foreground color
Hover	Background opacity increases to 90%
Animation	0.3s fade-in
Content	"Update v{X.Y.Z}" + SF Symbol
Visibility	Hidden when disabled or up-to-date

Integration point in `AetherApp.swift` (lines 48–51):

```
HStack {
    Spacer()
    if configManager.config.update.enabled {
        UpdateBadgeView(
            updateManager: updateManager,
            theme: configManager.config.colors
        )
        .padding(.trailing, 8)
    }
}
.frame(height: 28)
```

10.3 Update Modal

Centered overlay with glass morphism styling.



Figure 6: Modal states flow: Checking (spinner) leads to Downloading (progress bar), which leads to Ready to Install (restart/later buttons) or Failed (retry/dismiss). Installing shows a non-dismissable spinner.

Property	Value
Background	VisualEffectView(.hudWindow) + dark overlay
Corner radius	12pt
Width	320pt fixed
Height	Intrinsic (content-driven)
Buttons	Capsule, theme-aware colors
Progress bar	Capsule, accent fill, secondary track
Animation	Spring entrance, opacity exit
Dismiss	Click outside (not during install)

10.4 State-Driven Modal Content

State	Content
.checking	Spinner + “Checking...”
.downloading	Progress bar + size + Cancel
.readyToInstall	“Restart Now” + “Later”
.installing	Spinner + “Do not quit”
.failed	Error + Retry + Dismiss

10.5 Post-Update “What’s New” View

Shown after restart from an update. Reuses StartupView’s visual language.

1. **Stage 1:** OKernel logo (2.5s, same as first-run)
2. **Stage 2:** “What’s New in Aether {version}” — changelog with glass styling
3. **[CONTINUE]** button leads to normal terminal

Trigger: Check pendingUpdateVersion in UserDefaults on launch. If present, show this flow. Clear after display.

11. File System Layout

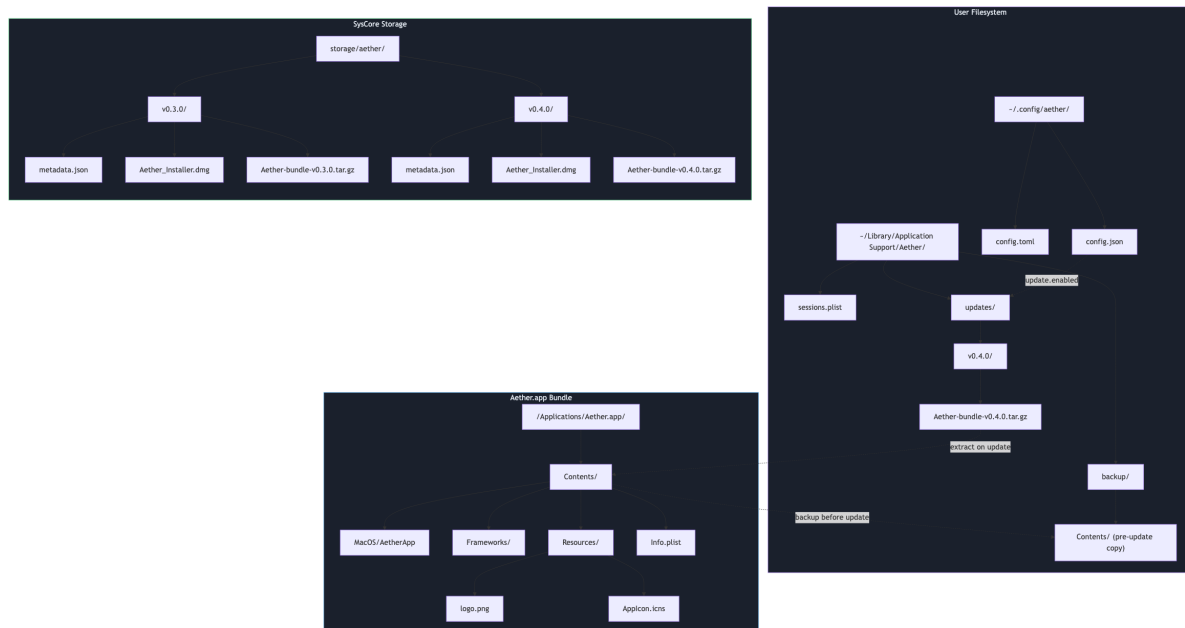


Figure 7: File system layout across three locations: user config (~/config/aether), application support (downloads and backups), and the Aether.app bundle that gets patched in-place.

```
~/Library/Application Support/Aether/
sessions.plist      (existing)
updates/
  v0.4.0/
    Aether-bundle-v0.4.0.tar.gz
  backup/
    Contents/      (pre-update copy)

~/config/aether/
config.json        (gains "update")
config.toml

/Applications/Aether.app/
Contents/
  MacOS/AetherApp  (replaced)
  Frameworks/     (replaced)
  Resources/      (replaced)
  Info.plist       (updated)
```

Directory Structure

12. Security Considerations

Concern	Mitigation
MITM	HTTPS via <code>api.hackmist.tech</code> . ATS enforced.
Tampered binary	Verify size vs <code>metadata.size</code> . Phase 2: SHA-256 checksum.
Privilege escalation	No <code>sudo</code> . Writes only to own bundle + user dirs.
Downgrade attack	Only update if <code>remote > local</code> (semver).
Interrupted install	Backup exists. Restore on failure.
Disk space	Check before download (need 2x update size).
Read-only bundle	Detect. Show: “Move Aether to writable location.”
Running from DMG	Detect read-only FS. Show: “Install Aether first.”

13. State Persistence

Key	Type	Purpose
<code>pendingUpdateVersion</code>	String?	Set before restart
<code>pendingUpdateChangelog</code>	String?	Changelog for “What’s New”
<code>lastUpdateCheck</code>	Date?	Last successful check
<code>updateDownloadedVersion</code>	String?	Completed download

14. Implementation Phases

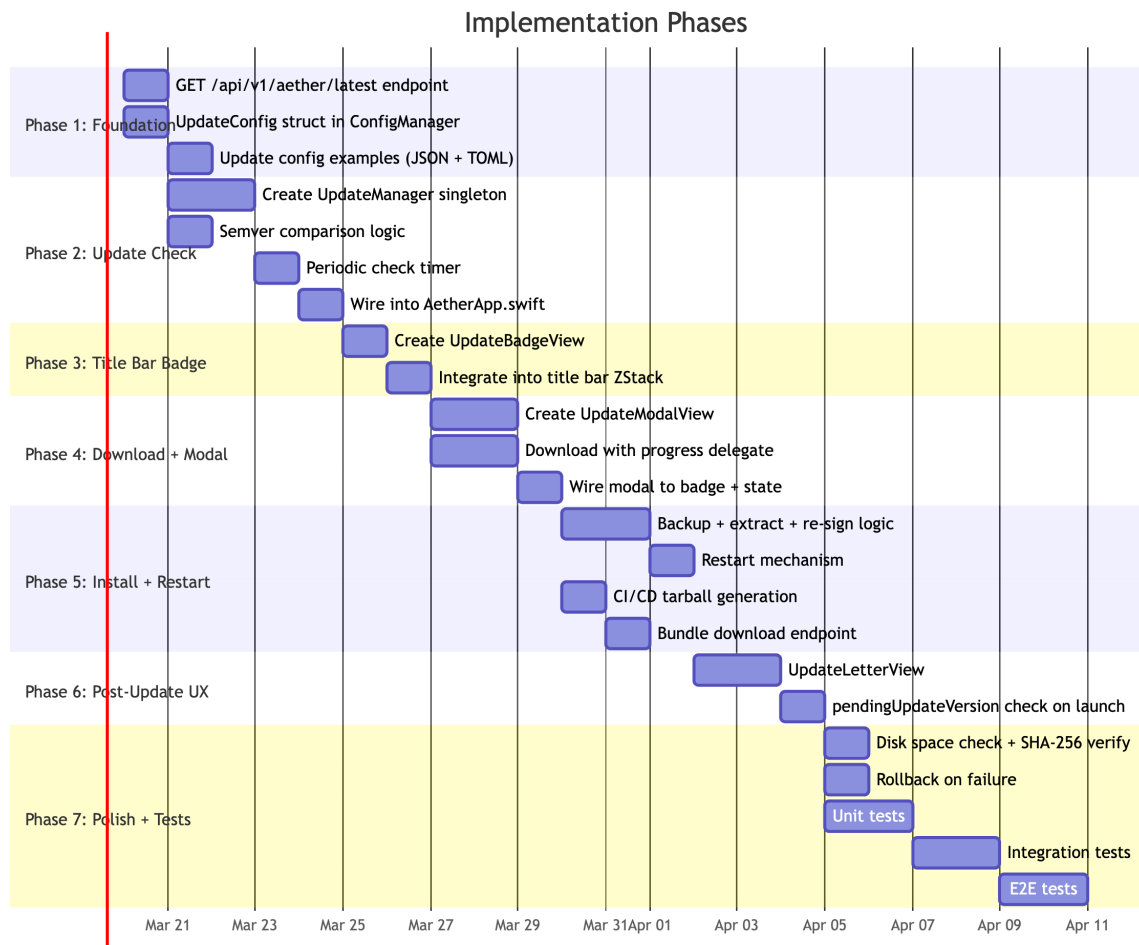


Figure 8: Gantt chart of 7 implementation phases: Foundation, Update Check, Title Bar Badge, Download + Modal, Install + Restart, Post-Update UX, and Polish + Tests.

14.1 Phase 1: Foundation

1. Add GET /api/v1/aether/latest to aether.rs
2. Add UpdateConfig struct to ConfigManager.swift
3. Update config.example.json and .toml
4. Update config decoder with defaults

14.2 Phase 2: Update Check

5. Create UpdateManager.swift singleton
6. Implement checkForUpdates() with URLSession
7. Implement semver comparison
8. Implement schedulePeriodicCheck() with Timer
9. Wire into AetherApp.swift on launch

14.3 Phase 3: Title Bar Badge

10. Create UpdateBadgeView.swift

11. Integrate into title bar ZStack
12. Bind to `UpdateManager.state`

14.4 Phase 4: Download + Modal

13. Create `UpdateModalView.swift`
14. Implement download with progress delegate
15. Implement `cancelDownload()`
16. Wire modal to badge tap + state

14.5 Phase 5: Install + Restart

17. Implement `applyUpdate()` (backup, extract, re-sign)
18. Implement `restartApp()` (`NSWorkspace` + terminate)
19. Add tarball generation to CI/CD
20. Add bundle download endpoint

14.6 Phase 6: Post-Update Experience

21. Create `UpdateLetterView.swift`
22. Check `pendingUpdateVersion` on launch
23. Show logo then changelog sequence

14.7 Phase 7: Polish + Tests

24. Disk space check before download
25. Backup restoration on failure
26. SHA-256 checksum verification
27. Auto-cleanup of old downloads/backups
28. Write all unit, integration, and E2E tests

15. Test Plan

15.1 Unit Tests

File: `Tests/AetherAppTests/UpdateManagerTests.swift`

ID	Verifies
U1	<code>v1.0.0 > v0.9.9</code> (major)
U2	<code>v0.4.0 > v0.3.0</code> (minor)
U3	<code>v0.3.1 > v0.3.0</code> (patch)
U4	<code>v0.3.0 == v0.3.0</code> (no update)
U5	<code>v0.2.0 < v0.3.0</code> (no downgrade)
U6	Handles <code>v</code> prefix and bare versions
U7	Default <code>UpdateConfig.enabled = true</code>
U8	Disabled config prevents checks
U9	Interval <code>< 300</code> clamped to 300
U10	Missing update key uses defaults
U11	TOML <code>[update]</code> parses correctly
U12	Full state machine transitions
U13	Progress calculation correctness
U14	Backup path construction
U15	Download path construction

File: `syscore/tests/api_tests.rs`

ID	Verifies
U16	/latest returns newest by date
U17	/latest returns 404 when empty
U18	Response has all required fields
U19	Bundle download serves gzip
U20	?v=latest resolves correctly

15.2 Integration Tests

File: Tests/AetherAppTests/UpdateIntegrationTests.swift

ID	Verifies
I1	Mock server check transitions to .available
I2	Same version stays .idle
I3	Unreachable server: no crash, retries
I4	Malformed JSON: graceful handling
I5	Download lands at correct path + size
I6	Cancel cleans up partial file
I7	Progress callbacks fire 0.0 to 1.0
I8	Runtime toggle invalidates timer
I9	Pending update persisted in UserDefaults
I10	Backup + corrupt + restore works

15.3 End-to-End Tests

File: Tests/AetherAppTests/UpdateE2ETests.swift

Prerequisites: Local SysCore on 127.0.0.1:3001, two test .tar.gz bundles (v0.3.0 and v0.4.0).

15.3.1 test_full_update_lifecycle

1. **Setup:** Upload v0.3.0 and v0.4.0 to SysCore. Create temp .app bundle at v0.3.0.
2. **Check:** checkForUpdates() transitions to .available("v0.4.0")
3. **Download:** downloadUpdate() completes; file exists with correct size
4. **Install:** applyUpdate() updates Info.plist to v0.4.0; backup exists
5. **Verify:** pendingUpdateVersion set; "What's New" would trigger
6. **Cleanup:** Delete temp dirs and test versions

15.3.2 test_update_with_disabled_config

1. Set update.enabled = false
2. Call checkForUpdates()
3. Assert: zero network requests (mock URLProtocol)
4. Assert: state == .idle, badge not visible

15.3.3 test_update_rollback_on_failure

1. Create temp bundle (v0.3.0), download v0.4.0
2. Make bundle path read-only (simulate failure)
3. applyUpdate() results in .failed
4. Backup restored — bundle still v0.3.0

15.3.4 test_interrupted_download_cleanup

1. Start download, cancel at 50%
2. Partial file cleaned up
3. Restart download from scratch succeeds

16. Risks and Mitigations

Risk	P	I	Mitigation
Read-only bundle path	M	H	Check write permission. Show error.
Running from DMG	L	H	Detect RO filesystem.
Partial extraction	L	H	Staging dir + atomic swap + backup.
Misconfigured URL	L	M	Validate on load. Log.
Slow connection	M	M	Show speed/ETA. Cancel/retry.
Concurrent updates	L	M	File lock in updates dir.

P = Probability (L/M/H), I = Impact (L/M/H)

17. File Change Summary

File	Act	Description
aether.rs	MOD	Add latest_handler, bundle download
lib.rs	MOD	Register new routes
ConfigManager.swift	MOD	Add UpdateConfig
UpdateManager.swift	NEW	Core update singleton
UpdateBadgeView.swift	NEW	Title bar pill button
UpdateModalView.swift	NEW	Download/install modal
UpdateLetterView.swift	NEW	“What’s New” screen
AetherApp.swift	MOD	Wire manager, badge, modal
StartupView.swift	MOD	Add update letter stage
config.example.json	MOD	Add update section
config.example.toml	MOD	Add [update] section
aether-release.yml	MOD	Tarball generation
UpdateManagerTests	NEW	Unit tests (U1–U15)
UpdateIntegrationTests	NEW	Integration (I1–I10)
UpdateE2ETests	NEW	End-to-end tests
api_tests.rs	MOD	Backend tests (U16–U20)